

Sorting 3

“

If you are insecure, guess what? The rest of the world is too. Do not overestimate the competition and underestimate yourself. You are better than you think.

T. HARVEY

GRACIOUSQUOTES.COM



Good
Evening

Today's content

01. Largest Number
02. Custom comparator on class
03. Nuts & Bolts
04. Partition of array
05. Quick sort
06. Count sort (if time left)

Largest Number

Given an array of integers, arrange them in such a way that they form the largest number when concatenated

Note:- The result may be very large, so return a string instead of an integer

Constraints

$$1 \leq n \leq 10^5$$

$$0 \leq A[i] \leq 10^8$$

$$A = \{2, 3, 9, 0\} \longrightarrow \text{Ans} = 9320$$

$$A = \{88, 9\} \longrightarrow \text{Ans} = 988$$

$$A = \{ \underline{89}, 8 \}$$

Two possible concatenations are shown:

- 898 ✓
- 889

→ Ans = 898

$$A = \{3, 30, 34, 5, 9\}$$

Two possible concatenations are shown:

- 9543330 ✗
- 9534330 ✓

Ideas $\Rightarrow$ Sort the data in descending order *

$\Rightarrow$ Sort the data based on digits in the descending order *

$A = \{37, 90\}$

Idea

$\{37, 90\}$ $\begin{cases} \rightarrow 3790 \rightarrow \text{string} \\ \rightarrow 9037 \rightarrow \text{string} \end{cases}$

$\{89, 8\}$ $\begin{cases} \rightarrow 898 \rightarrow \text{string} \\ \rightarrow 889 \rightarrow \text{string} \end{cases}$

```
Arrays.sort(ar, new Comparator<Integer>() {
```

```
    @Override
```

```
    public int compare(Integer a, Integer b)
```

```
    {
```

```
        String ab = "" + a + b;
```

```
        String ba = "" + b + a;
```

```
        Long abval = Long.parseLong(ab);
```

```
        Long baval = Long.parseLong(ba);
```

```
        if (abval < baval) return 1;
```

```
else if (bval < abval) return -1;
```

```
else return 0;
```

```
});
```

```
String str = " ";  
for (int v : ar) {  
    str = str + v;  
}
```

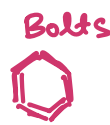
```
String str = " ";  
for (int i = 0; i < ar.length; i++)  
    int v = ar[i]  
    str = str + v;
```

Why usually the mod value = $10^9 + 7$
universal
Hashing collision

* Comparator for a class

Student class

Nuts & Bolts

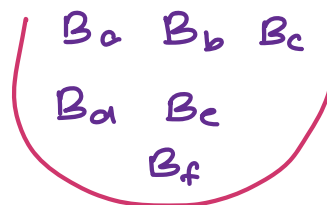
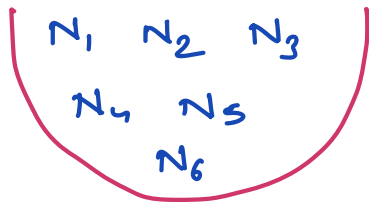


Given N nuts of different sizes & B bolts of different sizes.

There is a 1:1 mapping b/w nuts & bolts

Match the nut & bolts with a given constraint that comparing a nut with a nut & a bolt with another bolt is not allowed.

Compare a nut & bolt $\left\{ \begin{array}{l} \text{exactly fit} \\ \text{nut is small} \\ \text{nut is big} \end{array} \right.$



Sort both the buckets separately $\times$

Brute force $\rightarrow$ Pick a nut & compare it with all the bolts $\left\{ \begin{array}{l} \text{for each} \\ \text{\&} \\ \text{every} \\ \text{nut} \end{array} \right.$

No. of
iteration

$$6 + 5 + 4 + 3 + 2 + 1$$

$$Tc: O(n^2)$$

Idea 2

N_1

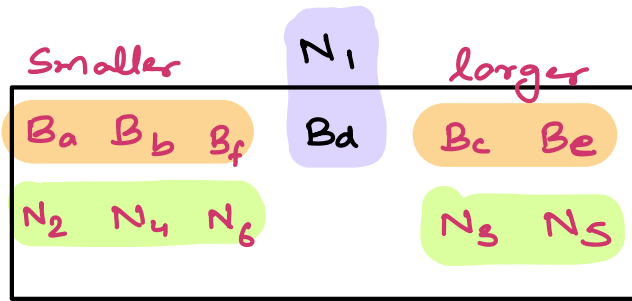
N_2 ✓

N_3 ✓

N_4 ✓

N_5 ✓

N_6 ✓



B_a ✓

B_b ✓

B_c ✓

B_d ✓

B_e ✓

B_f ✓

Partition → Dividing our data into two parts by using a **pivot** ele

Q Given an integer array, rearrange the elements in it such that for all i ,

if $A[i] < x$ then it should be present on **left side**

if $A[i] \geq x$ then it should be present on **right side**

$$A = \{ 9 \ 8 \ 1 \ 6 \ 5 \ 8 \} \quad x = 6$$

$$\text{Ans} = \{ 1 \ 5 \ 6 \ 9 \ 8 \ 8 \}$$

$$\text{Ans} = \{ 5 \ 1 \ 6 \ 8 \ 9 \ 8 \}$$

$$\text{Ans} = \{ 1 \ 5 \ 6 \ 8 \ 8 \ 9 \}$$

All the ele $< x$
must be on left &
ele $> x$ must be
on to RH

Idea 1 $\rightarrow$ Sort the array $\Rightarrow \{1, 5, 6, 8, 8, 9\}$

TC: $O(n \log n)$

Idea 2 $\rightarrow$ Use two pointer approach

$j \rightarrow$ To traverse on the array

$i \rightarrow$ indicate the place where we have to insert the smaller ele

$A = \{ \cancel{9}, \cancel{8}, \cancel{1}, \cancel{6}, \cancel{5}, 8 \} \quad x=7$

$\begin{matrix} & & & & & j \\ & & & & & \downarrow \\ & 1 & 6 & 9 & 8 & 9 \\ & & & 5 & & \\ & & & \uparrow \\ & & & i \end{matrix}$

$A = \{1, 6, 5, 8, 9, 8\}$

Small no. $< x = [0, i-1]$

Larger no. $\geq x = [i, n-1]$
or equal no

$A = \{ 3, 5, 7, 0, 2, 8 \}$ $x = 7$
 (Annotations: j points to 7, i points to 2, with intermediate steps showing 7 moving to index 2)

$$\begin{cases} \text{arr}[j] < x \\ \text{swap}(i, j) \end{cases}$$

$A = \{ 3, 5, 0, 2, 7, 8 \}$

$i = 0$

for ($j = 0; j < N; j++$)

if ($A[j] < x$) {

swap($A[i], A[j]$)

$i++;$

$$\begin{cases} \text{TC: } O(n) \\ \text{SC: } O(1) \end{cases}$$

3

10:29 → 10:39 pm

$\{ 7, 8, 3 \}$ $x = 6$
 (Annotations: j points to 3, i points to 3, with a blue '3' above the first 3)

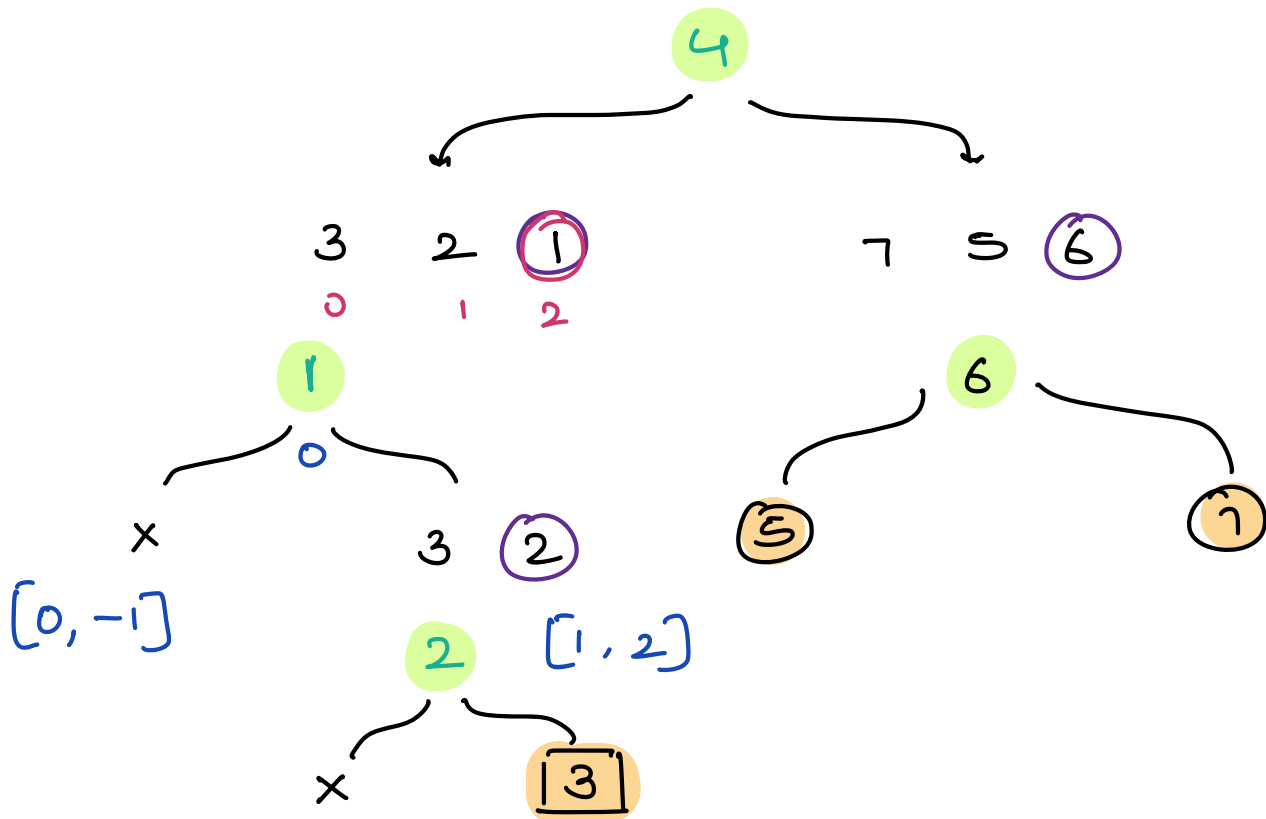
$3 < 6$

Quick sort

arr =

7	3	2	5	1	6	4
0	1	2	3	4	5	6

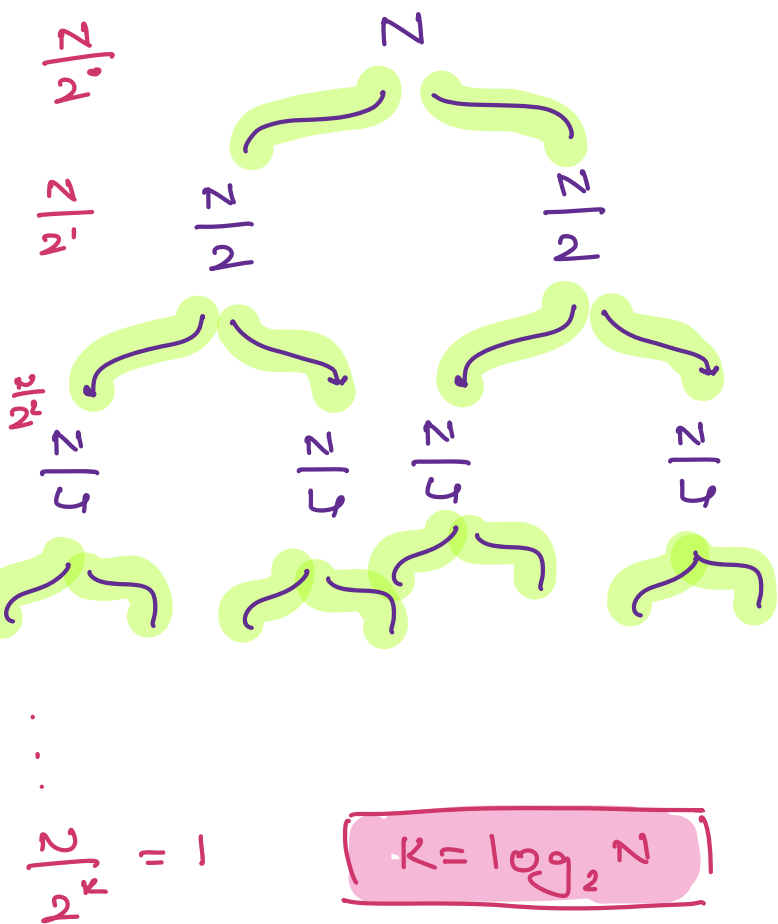
Pivot = 4



Ans = { 1, 2, 3, 4, 5, 6, 7 }

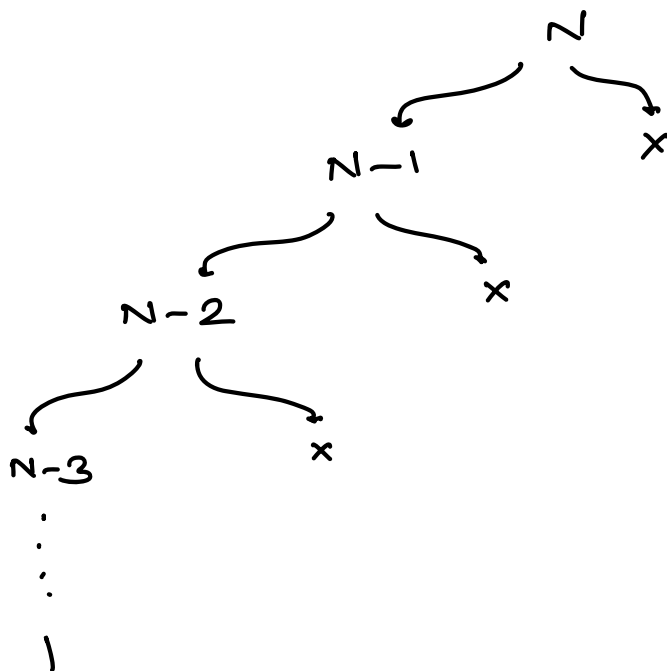
Idea → Pick a pivot ele, move it to correct position & call for the subproblem
 $[0 \text{ to } i-1]$ & $[i+1 \text{ to } n-1]$

Time complexity $\propto$



Arr =

7	6	5	4	3
---	---	---	---	---



$K = n$

Since we are doing partitioning at each & every step $TC: O(N)$

$TC: O(n \log n)$ $\leq$ TC for quick Sort $\leq$ $TC: O(n^2)$

Best Worst

For 99.9% = $TC: O(n \log n)$

```
void quicksort (int [] A, int s, int e)
```

```
    if (s ≥ e) return;
```

```
    int pi = partition (A, s, e)
```

```
    quicksort (A, s, pi - 1)
```

```
    quicksort (A, pi + 1, e)
```

3

```
int partition (int [] A, int st, int end)
```

```
    int p = A[end]
```

```
    int i = st
```

```
    for (j = st ; j < end ; j++) {
```

```
        if (A[j] < p) { ←
```

```
            swap(A[j], A[i]);
```

```
            i++;
```

3

3

```
    swap(A[i], A[end]);
```

```
    return i;
```

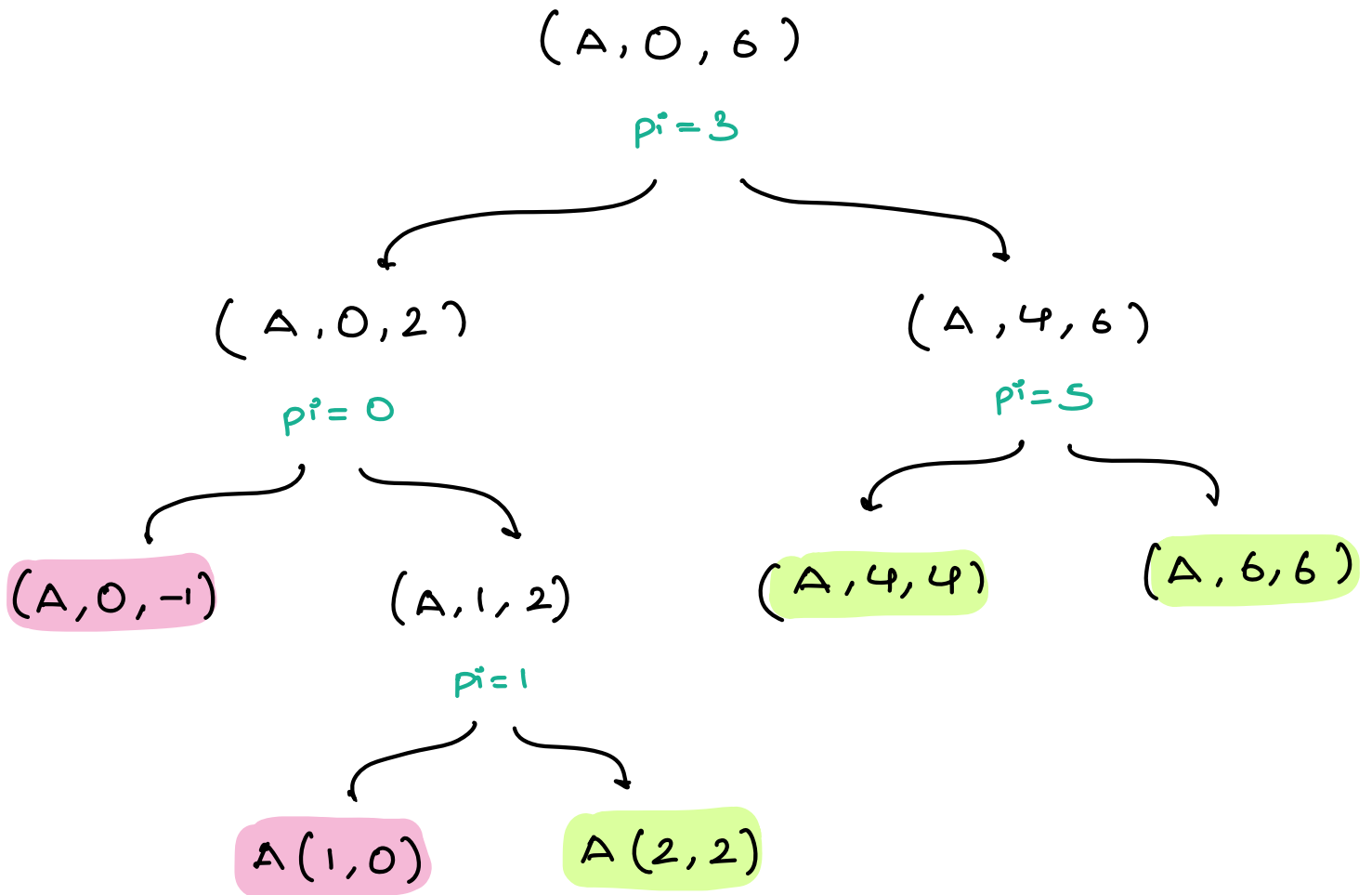
3

arr =

3	2	1	4	7	6	5
0	1	2	3	4	5	6

arr =

7	3	2	5	1	6	4
0	1	2	3	4	5	6



Merge Sort

Quicksort

Time

$O(n \log n)$

$O(n^2) \rightarrow \text{Worst}$

Space

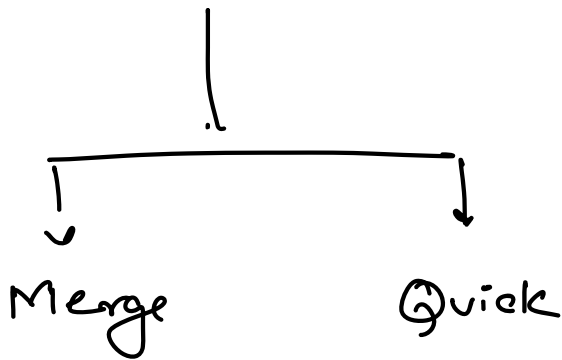
$O(n)$

$O(1)$

Arrays. sort

TC : $O(n \log n)$

SC : $O(1)$



* **Count Sort** ↗ Range = small

↳ Given an array of **digits**, we need to form the smallest no. possible

arr =

7	3	1	0	8	3	1
---	---	---	---	---	---	---

0 1 2 3 4 5 6

Ans = **0 1 1 3 3 7 8**

TC: $O(n)$

SC: $O(\text{range})$

Range = 0 to 9

Countarr =

1	2	0	2	0	0	0	1	1	0
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 9

Iterate on countarr & print the index

↓
freq no. of times

Ans = 0 1 1 3 3 7 8

10^7	1
--------	---

→ Count sort works better with small range

{ TO DO }

* Post lecture of Recursion 3



Tower of Hanoi

Doubts

Max chunk { 1, 2, 0, 3, 5, 4 }

~~~~~

0 1 2 3 4 5

Ans = 3

Max chunk = { <sup>0 1 2 3</sup> 2 3 0 1 | <sup>4</sup> 4 | <sup>5</sup> 5 | <sup>6 7</sup> 7 6 }

[ 0 1 2 3 ]

Smallest left chunk =

Max { 1, 3, 0, 2, 5, 4 }

0 1 2 3 4 5

↑

$i = 0 \rightarrow n$

$max = Math.max(max, A[i])$

if ( $max == i$ ) chunk ++